Task 2 Roadmap

Build and deploy a Heart Disease Prediction System by copying the Titanic project step by step.

Deadline: The assignment deadline was Sunday 11 Oct 2026 at 23:59 and the 3-day cutoff has passed. Message Dr. Rao / check Google Classroom to see if a late submission is still accepted. Everything below is designed so you can finish quickly.

1. The big picture

Task 2 is the Titanic project again, with a different dataset. You do **not** invent a new design. You swap the data and the column names, keep every step, and deploy the result as a web app. I built and tested the notebook and app for you (files sent with this PDF). This guide tells you what each step does, what you should see, and how to deploy.

Stage	What happens	File produced
Download	Get the UCI Heart Disease data	processed.cleveland.data
Understand + clean	Look at the data, keep 4 columns, fix the target	-
Feature extraction	Turn numbers into readable words (Male, Low...)	heart-data.csv
Label encoding	Turn words into numbers the model can use	heart-data-encoded.csv
Training	80% of rows teach the SVC model	heart_disease_svc_model.pkl
Testing	20% of rows check it (accuracy, confusion matrix)	model-predictions.csv
Application	Predict for one new patient	-
Deployment	Streamlit app on the internet + link at top of notebook	app.py, requirements.txt

2. Titanic vs Heart Disease: what changes

Part	Titanic	Heart Disease
Raw data	sample-data.csv (100 rows, already clean words)	processed.cleveland.data (303 rows, numbers, no header, '?' = missing)
Inputs (4)	PClass, Gender, Sibling, Embarked	Sex, ChestPain, ExerciseAngina, MaxHeartRate
Target (last column)	Survived: Yes / No	HeartDisease: Yes / No
Pre-processing	Nothing to do	Keep 5 columns, drop missing, make target Yes/No
Feature extraction	Nothing to do	Real work: numbers become words, heart rate grouped
Encoders	5 LabelEncoders	5 LabelEncoders (same idea, new names)
Split	80/20, no shuffle	80/20 with shuffle and stratify (file is not pre-mixed)
Model	SVC(gamma='auto', random_state=0)	Exactly the same
Saved model	svc_trained_model.pkl	heart_disease_svc_model.pkl
Deployment	streamlit-app/app.py	app.py (same pattern, 4 new dropdowns)

3. The 4 input attributes I chose (and why)

Attribute	UCI column	Becomes (words)	Why it is good
Sex	sex	Male, Female	Simple, no missing values
ChestPain	cp	Typical angina, Atypical angina, Non-anginal pain, Asymptomatic	One of the strongest signals in this dataset
ExerciseAngina	exang	Yes, No	Chest pain during exercise is a strong warning sign
MaxHeartRate	thalach	Low (under 140), Medium (140-159), High (160+)	Strong signal; grouping keeps the Titanic code unchanged
HeartDisease (target)	num	Yes, No	0 = No, anything 1 to 4 = Yes

Why group the heart rate? The Titanic code uses LabelEncoder and SVC(gamma='auto'). SVC works badly when one column has big numbers (heart rate around 150) next to small ones (0 to 3). Grouping heart rate into Low/Medium/High avoids that problem and keeps all 4 inputs in the same word-to-number style as Titanic. **If your teacher asks why:** the 3 groups were cut near the 1/3 and 2/3 points of the data.

4. Phase A: Set up (10 minutes)

1. **Download the data.** Open https://archive.ics.uci.edu/dataset/45/heart+disease and click **Download**. Unzip it. You will see several files. You need only **processed.cleveland.data** (303 rows). Ignore the others.
2. **Make a project folder**, for example `heart-disease-task2`, and put these 2 files in it: `Heart_Disease_Prediction.ipynb` and `processed.cleveland.data`.
3. **Use the same Python environment you used for Titanic** (the one where you ran `pip install -r requirements.txt` from the teacher's repo). It already has pandas, scikit-learn, seaborn, matplotlib and prettytable.
4. Open a terminal in the folder and run `jupyter notebook`. Open `Heart_Disease_Prediction.ipynb`.

```
heart-disease-task2/
  Heart_Disease_Prediction.ipynb
  processed.cleveland.data        <- from UCI
```

Write this down: Run `pip show scikit-learn` and note the version. The notebook also prints it in Step 1. The version you train with must be the version in requirements.txt on the website, otherwise the app can fail to load your model.

5. Phase B: The notebook, step by step

Run the cells from top to bottom (Kernel, then Restart & Run All). The notebook is already filled in. Below is what each step does and what you should see on the real file.

Do not forget: Cell 1 of the notebook is the line **Live Deployment Link: PASTE_YOUR_STREAMLIT_APP_LINK_HERE**. Replace it with your real link after you deploy. The assignment requires it at the very top. Step 10 has a second placeholder to replace as well.

Step 1: Import libraries

Same as Titanic. It also prints your scikit-learn version.

Step 2: Load the data

The UCI file has no header row and writes missing values as '?'. So we give the column names ourselves and tell pandas that '?' means missing.

```
COLUMN_NAMES = ["age","sex","cp","trestbps","chol","fbs","restecg",
                "thalach","exang","oldpeak","slope","ca","thal","num"]
raw_data = pd.read_csv("processed.cleveland.data", header=None,
                       names=COLUMN_NAMES, na_values="?")
```

Step 3.1: Understand the data

The notebook prints shape, column types, missing values and the target counts. On the real file you should see:

- Shape **(303, 14)**.
- Missing values: **ca = 4**, **thal = 2**, everything else 0. We do not use those two columns.
- Target **num** counts: 0 = 164, 1 = 55, 2 = 36, 3 = 35, 4 = 13.

Step 3.2: Pre-process

Titanic said 'nothing to do'. Here we do three small things: keep 4 inputs + target, drop rows with missing values (none in our columns), and make the target Yes/No by treating 1, 2, 3, 4 all as 'disease'.

```
selected_data = raw_data[["sex","cp","exang","thalach","num"]].copy()
selected_data = selected_data.dropna()
selected_data["num"] = (selected_data["num"] > 0).astype(int)
```

Expected: rows dropped = **0**. Target becomes **0 = 164** and **1 = 139**.

Step 4: Feature extraction

Titanic's data was already in words. Heart Disease is in numbers, so we convert them to words, group the heart rate into Low/Medium/High, and put the target last. The result is saved as `heart-data.csv` (this is the clean dataset you can submit).

```
heart_data = pd.DataFrame()
heart_data["Sex"] = selected_data["sex"].astype(int).map({1: "Male", 0: "Female"})
heart_data["ChestPain"] = selected_data["cp"].astype(int).map({
    1: "Typical angina", 2: "Atypical angina",
    3: "Non-anginal pain", 4: "Asymptomatic"})
heart_data["ExerciseAngina"] = selected_data["exang"].astype(int).map({1: "Yes", 0: "No"})
heart_data["MaxHeartRate"] = pd.cut(selected_data["thalach"], bins=[0, 139, 159, 400],
                                    labels=["Low", "Medium", "High"]).astype(str)
heart_data["HeartDisease"] = selected_data["num"].map({1: "Yes", 0: "No"})   # target LAST
```

Step 4.1 draws 4 small bar charts (heart disease rate per group). Use them in your conclusion: for example, 'Asymptomatic' chest pain and exercise angina should show noticeably higher disease rates.

Step 5: Label encoding

Same idea as Titanic. Each column gets its own LabelEncoder, trained on the list of all possible words. `np.ravel()` flattens the one-column table into a plain list. The notebook then prints which number each word got.

```
sex = pd.DataFrame({"Sex": ["Male", "Female"]})
chest_pain = pd.DataFrame({"ChestPain": ["Typical angina", "Atypical angina",
                                         "Non-anginal pain", "Asymptomatic"]})
exercise_angina = pd.DataFrame({"ExerciseAngina": ["Yes", "No"]})
max_heart_rate = pd.DataFrame({"MaxHeartRate": ["Low", "Medium", "High"]})
heart_disease = pd.DataFrame({"HeartDisease": ["Yes", "No"]})

sex_label_encoder = LabelEncoder()          # ...one per column...
sex_label_encoder.fit(np.ravel(sex))        # ...and fit each one like this
```

Column	Number codes (alphabetical order, you should see the same)
Sex	Female = 0, Male = 1
ChestPain	Asymptomatic = 0, Atypical angina = 1, Non-anginal pain = 2, Typical angina = 3
ExerciseAngina	No = 0, Yes = 1
MaxHeartRate	High = 0, Low = 1, Medium = 2
HeartDisease	No = 0, Yes = 1

Steps 5.2 and 5.3 encode the target first and then the 4 inputs, and save `heart-data-encoded-output.csv` and `heart-data-encoded.csv`.

Step 6: Training

```
training_data_encoded, testing_data_encoded = train_test_split(
    heart_data_encoded, test_size=0.2, random_state=0,
    shuffle=True, stratify=heart_data_encoded["HeartDisease"])
```

Differences from Titanic: **shuffle=True** because the UCI file is not pre-mixed, and **stratify** keeps the Yes/No share equal in train and test. Expected shapes: training **(242, 5)**, testing **(61, 5)**. Then 6.2 splits inputs/target with `iloc[:, :-1]` and `iloc[:, -1]` (this is why the target must be the last column), 6.3 trains `svm.SVC(gamma='auto', random_state=0)`, and 6.4 saves `heart_disease_svc_model.pkl`.

Step 7: Testing

Loads the .pkl, predicts the 61 test rows, then prints accuracy, the classification report (precision, recall, F1) and a confusion matrix heatmap.

- **What is a normal accuracy?** With only 4 inputs, expect roughly the 70s to low 80s percent. I could not download the UCI file in my workspace, so I could not measure it on the real data. Whatever you get, write

that number in your conclusion.

- **Red flags:** exactly 100% (the target leaked into the inputs) or around 50% (the columns got mixed up). Re-run from the top if you see these.

Step 8: Application phase (one new patient)

Edit the 4 demo values, run the cells, and the notebook prints the prediction. This is exactly the path the web app follows: words, then a one-row table, then encoders, then the model.

```
sex_input = "Male"
chest_pain_input = "Asymptomatic"
exercise_angina_input = "Yes"
max_heart_rate_input = "Low"
```

Steps 9 and 10

Step 9 (feedback) is illustrative text, clearly marked as sample entries. After deployment, ask 2 or 3 classmates to try the app and replace the sample rows with their real comments. Step 10 describes the deployment. Finish with your own 2 or 3 sentence conclusion.

6. Phase C: The web app files

Make a new folder for the website with these 4 files (I generated all of them for you, except the model, which your notebook creates):

```
heart-disease-app/
  app.py                          <- the web page (sent with this PDF)
  heart_disease_svc_model.pkl     <- created by YOUR notebook run (Step 6.4)
  requirements.txt                <- the packages the website installs
  runtime.txt                     <- Python version note (3.11)
```

Three things to check before you upload:

1. **Copy the model.** Copy `heart_disease_svc_model.pkl` from your notebook folder into this folder. Do this again every time you re-run Step 6.4.
2. **Match the scikit-learn version.** Open requirements.txt. The line `scikit-learn==1.7.2` must equal the version your notebook printed in Step 1. If yours is different, change the number. (Teacher's repo uses 1.7.2, so if you used his requirements file you will match.)
3. **Keep the word lists identical.** The lists at the top of app.py must use exactly the same words as Step 5.1 of the notebook. I already copied them. Only touch them if you change categories.

```
streamlit==1.64.0
scikit-learn==1.7.2
pandas==2.3.3
numpy==2.2.6
```

Optional: after you get your accuracy, set `TEST_ACCURACY = 0.78` (your number) near the top of app.py and the app will show it.

Test it on your computer first

```
pip install streamlit
streamlit run app.py
```

A browser tab opens at http://localhost:8501. Pick values, click **Predict**, and check you get 'HEART DISEASE LIKELY' or 'NOT LIKELY' without an error. The patient from Step 8 (Male, Asymptomatic, Yes, Low) should be 'LIKELY'.

> **Careful:** Installing a newer Streamlit locally is fine, but your scikit-learn must stay at the version used for training. If pip changes it, reinstall the right one: `pip install scikit-learn==YOUR_VERSION`.

7. Phase D: Deploy on Streamlit Community Cloud (free)

1. **Create a GitHub account** (if you do not have one) and a **new public repository**, for example `heart-disease-app`.
2. On the repo page click **uploading an existing file**, drag in the 4 files (app.py, heart_disease_svc_model.pkl, requirements.txt, runtime.txt) and click **Commit changes**. Make sure the files sit at the top level of the repo, not inside another folder.
3. Go to **https://share.streamlit.io** and sign in with GitHub.
4. Click **Create app** and choose to deploy from an existing GitHub repo.
5. Fill in: Repository = your repo, Branch = `main`, Main file path = `app.py`.
6. Open **Advanced settings** and choose **Python 3.11**. (The Titanic repo's history shows the default newest Python lacked scikit-learn builds and broke the install.)
7. Click **Deploy**. The first build takes a minute or two. If it hangs, open **Manage app** and read the log.
8. Copy the address, which looks like `https://your-name-heart-disease-app.streamlit.app`. Open it in a private window to confirm it works and anyone can reach it.
9. Paste that link into **the very first cell of the notebook** and into Step 10.

Note: Button names on the website can change slightly over time, but the 3 values (repo, branch main, main file app.py) and the Python version are what matter.

Task 1 tip: If you have not deployed Titanic yet, the quickest way is the same process: fork the teacher's repo (github.com/haqnawaz99/ml-teaching-notebooks) to your account, then in Streamlit Cloud choose your fork, branch main, main file `streamlit-app/app.py`.

8. Phase E: Submit on Google Classroom

Upload	File	Done?
Code	Heart_Disease_Prediction.ipynb (run with outputs visible; live link in the first cell)	[ ]
Dataset	processed.cleveland.data (original) and heart-data.csv (cleaned version)	[ ]
Trained model	heart_disease_svc_model.pkl	[ ]
Optional extras	app.py, requirements.txt (shows your deployment setup)	[ ]
Task 1	Titanic notebook + dataset + model + its live link, as the assignment asks	[ ]

Before uploading the notebook: Kernel, then **Restart & Run All**, then save, so the outputs match the final code. Check that the first cell holds your real link.

9. If something breaks

Problem	Fix
FileNotFoundError: processed.cleveland.data	The data file must be in the same folder as the notebook. Check the name (no .txt added by Windows).
Step 4 assertion: 'Some values could not be converted'	You probably used a different UCI file. Use processed.cleveland.data and run from Step 2.
ValueError: y contains previously unseen labels	A word in your data or app is not in the Step 5.1 list. Check spelling and capital letters.
App error: model file not found	heart_disease_svc_model.pkl is missing from the repo or is inside a sub-folder.
Warning or error loading the model on the website	scikit-learn version in requirements.txt differs from the one you trained with. Fix the number, re-run Step 6.4 if needed, upload again.

Problem	Fix
'Your app is in the oven' for a long time	Open Manage app, read the log. Usually a typo in requirements.txt or the wrong Python version.
Predictions always the same	Normal for very different inputs to give the same answer sometimes, but if *every* input gives the same result, re-run the notebook from the top and redo the split.

10. Questions you may be asked

- **Why 4 attributes?** The assignment limits us to 4. We chose the ones that are complete (no missing values) and informative.
- **Why make the target Yes/No?** The raw column has 0 to 4. We predict 'disease present or not', which is binary like Titanic's survived or not.
- **Why label encoding?** Machine learning models need numbers, not words.
- **Why save a .pkl?** So the app can reuse the trained model without training again.
- **Why shuffle and stratify?** The file is not pre-mixed, and stratify keeps the Yes/No share equal in train and test.
- **What does the confusion matrix show?** How many patients of each real class were predicted correctly or wrongly.
- **Why is the app not medical advice?** Only 4 inputs and about 300 patients. It is a learning project.